Data Storage Architecture in the Phase-1 Blockchain E-Voting System

The Phase-1 blockchain e-voting system is designed as a hybrid architecture that combines a traditional backend application with a decentralized blockchain ledger. Each layer of the system — the blockchain, the backend infrastructure, and the frontend application — serves a distinct purpose and stores different categories of data. This separation is essential because blockchain networks are not suitable for storing large volumes of application data or personal information, while traditional databases cannot provide the immutability and transparency required for recording votes. By dividing storage responsibilities among these layers, the system achieves both operational efficiency and cryptographic integrity.

In this architecture, the blockchain acts as the immutable ledger responsible for enforcing election rules and recording votes. The backend database manages all operational data related to users, elections, and system workflows. The frontend serves as the user interaction layer and temporarily holds data required for wallet interaction and voting operations. Understanding what information belongs in each layer is critical to maintaining the security, scalability, and privacy guarantees of the system.

Data Stored on the Blockchain

The blockchain is responsible for storing the minimum set of information required to enforce the integrity of the election. Because every piece of data written to a blockchain requires computational resources and transaction fees, the system must limit on-chain storage to only those elements that must remain tamper-proof and publicly verifiable. The smart contract deployed for each election therefore contains the core voting logic and a small set of data structures necessary to guarantee correct voting behavior.

One of the most important elements stored on the blockchain is the set of election parameters defined when the election contract is deployed. These parameters include the start time and end time of the election, which determine the period during which voters are allowed to submit ballots. By storing these timestamps directly in the smart contract, the blockchain itself enforces the election schedule and automatically rejects any vote submitted outside the permitted window.

The smart contract also stores the identifiers of the candidates participating in the election along with a vote counter associated with each candidate. Each time a valid vote transaction is executed, the contract increments the vote counter corresponding to the selected candidate. Because these counters are stored directly on the blockchain, they cannot be altered by

administrators or backend systems once a vote has been recorded. This ensures that the final tally reflects the exact number of votes submitted during the election period.

Another critical component stored on the blockchain is the record of voter participation. The contract maintains a mapping that associates blockchain wallet addresses with a boolean flag indicating whether the address has already cast a vote. When the voting function is called, the contract checks this mapping before processing the transaction. If the address has previously voted, the contract rejects the transaction and prevents duplicate voting. This mechanism ensures that each blockchain address can participate only once in a given election.

In elections where eligibility is restricted to a predefined group of voters, the contract also stores the Merkle root representing the list of authorized voter identifiers. Instead of storing the entire voter list on the blockchain, which would be inefficient and expensive, the backend generates a Merkle tree from the list of eligible identifiers and stores only the root hash on the contract. When a voter submits a vote transaction, they include a Merkle proof that demonstrates their inclusion in the original eligibility set. The contract verifies this proof against the stored root to confirm that the voter belongs to the authorized group.

Finally, the contract maintains the state of the election itself. This state determines whether the election is active, completed, or locked from further voting. By storing the election state within the contract, the blockchain becomes the definitive authority on whether votes can still be accepted.

Through these mechanisms, the blockchain stores only the elements necessary to enforce voting rules and record the outcome of the election. No personal identity information, voter profiles, or administrative data should ever be written to the blockchain, as doing so would create privacy risks and unnecessary operational costs.

Data Stored in the Backend Database

While the blockchain stores the immutable record of voting activity, the backend database serves as the operational core of the system. The backend is responsible for managing the application’s business logic, user identities, administrative workflows, and election configuration. These types of data are better suited to a traditional database because they require frequent updates, complex queries, and privacy protection that cannot be easily implemented on a public blockchain.

One of the primary responsibilities of the backend is storing voter identity information collected during the registration process. When a user signs up for the platform, the backend records details such as their name, email address, phone number, residential address, and identifiers such as student ID or employee ID. In systems that use Aadhaar for identity anchoring, the Aadhaar number must be securely processed and stored in hashed form rather than in plaintext.

This ensures that sensitive personal information is protected while still allowing the system to enforce uniqueness and identity verification.

The backend also stores authentication data required for secure access to the system. This includes hashed passwords, session tokens such as JSON Web Tokens, and records related to OTP verification during the signup process. These mechanisms enable the platform to authenticate users and control access to both voter and administrator dashboards.

Another major category of data stored in the backend database is election metadata. When an administrator creates a new election, the backend records information such as the election title, description, organization, election type, visibility settings, and the scheduled start and end times. Once the election contract is deployed on the blockchain, the backend stores the resulting contract address and associates it with the election record. This contract address becomes the reference used by the frontend to interact with the smart contract.

Candidate information is also stored entirely within the backend database. Although the blockchain stores candidate identifiers and vote counts, it does not store descriptive information such as candidate names, party affiliations, symbols, manifestos, or profile images. The backend maintains this information so that the frontend can display candidate details on the ballot interface without writing unnecessary data to the blockchain.

For elections that require restricted eligibility, the backend stores the complete list of authorized voter identifiers before generating the Merkle tree. These identifiers may include student IDs, employee IDs, or other organizational credentials. After the administrator locks the eligibility list, the backend constructs the Merkle tree, computes the root hash, and stores both the root and the individual voter entries in the database. Only the root hash is written to the blockchain contract.

The backend also maintains records of vote transactions submitted by users. After a voter signs a vote transaction through their wallet, the frontend receives the resulting transaction hash and sends it to the backend. The backend records this hash along with the voter’s account ID, wallet address, election ID, and timestamp. These records allow the system to display confirmation messages, track participation statistics, and provide links to blockchain explorers where the transaction can be independently verified.

Additionally, the backend stores verification records indicating whether a voter has successfully completed the pre-verification process for a specific election. These records allow the system to display eligibility status on the voter dashboard and streamline the voting process when the election becomes active.

In summary, the backend database stores all application-level information necessary to operate the voting platform while avoiding the high costs and privacy risks associated with storing such data on the blockchain.

Data Stored in the Frontend Application

The frontend application functions primarily as the user interface through which voters and administrators interact with the system. Unlike the backend database or blockchain ledger, the frontend does not serve as a permanent storage layer. Instead, it temporarily holds data required for user interactions, wallet connections, and blockchain communication during a session.

One of the most important pieces of data handled by the frontend is the wallet connection state. When a voter connects their blockchain wallet to the application, the frontend retrieves and temporarily stores the wallet address associated with the session. This address is used when interacting with the election smart contract and is included as the sender of the vote transaction.

The frontend also maintains temporary user interface state, such as the election currently being viewed, the candidate selected by the voter, and the eligibility status returned by the backend. These pieces of information exist only while the user is interacting with the application and are discarded when the session ends or the page is refreshed.

Another important element managed by the frontend is the smart contract interface itself. The frontend loads the contract's Application Binary Interface (ABI) and uses the contract address retrieved from the backend to construct a connection to the election contract. This interface enables the frontend to call contract functions for voting or reading election results.

During the voting process, the frontend may also temporarily store the Merkle proof returned by the backend after eligibility verification. This proof is required when calling the voting function of the smart contract and is transmitted as part of the transaction. Once the transaction is completed, the proof is no longer needed and does not need to be stored permanently.

Finally, the frontend receives the transaction hash generated when the voter signs and submits the vote transaction. This hash is displayed to the user as confirmation that their vote has been recorded on the blockchain. The frontend then forwards the hash to the backend so it can be stored in the database for auditing and verification purposes.

Because the frontend operates within the user's browser environment, it should never store sensitive identity data or permanent records. Its primary role is to display information retrieved from the backend, facilitate wallet interactions, and relay transactions to the blockchain network.

Users Table (Voters and Admins)

The **Users table** stores all individuals who interact with the e-voting platform.
This includes both **voters** who participate in elections and **administrators** who manage election creation and configuration.

The system follows a **role-based access model**, meaning that different system capabilities are granted depending on the role assigned to the user. Administrators are responsible for creating and configuring elections, while voters are responsible for registering, verifying their identity, and casting votes.

The table also stores the **identity anchors and authentication information** required for secure participation in the voting system. These include verified contact information, Aadhaar-linked identity verification, and the blockchain wallet address used for signing voting transactions. Storing the wallet address allows the system to associate a user account with the blockchain address used to cast votes, enabling auditability while keeping the vote itself on the blockchain.

Because the platform supports **multiple organizations running elections within the same system**, administrator accounts are associated with a specific organization. This ensures that administrators can only manage elections belonging to their organization.

Sensitive information such as Aadhaar numbers and passwords must **never be stored in plain text**. Instead, the system stores **hashed values** to protect user privacy and maintain security compliance.

Field Name	Data Type	Description
user_id	UUID (Primary Key)	Unique identifier assigned to each user in the system.
full_name	VARCHAR	The user's full legal name used for identification within the platform.
email	VARCHAR (Unique)	Email address used for login and communication with the platform.
password_hash	VARCHAR	Securely hashed password used for authentication. Plain text passwords are never stored.
phone_number	VARCHAR	Phone number used for OTP-based identity verification during signup and login.

aadhaar_hash	VARCHAR	Cryptographic hash of the user's Aadhaar number used for identity anchoring and duplicate account prevention.
role	ENUM ('admin','voter')	Defines the role of the user in the system. Determines access permissions and system functionality.
wallet_address	VARCHAR	Blockchain wallet address associated with the user. Used when signing vote transactions through the wallet.
organization_id	UUID (Foreign Key)	References the organization the user belongs to. Primarily used for administrators managing elections for a specific organization.
student_id	VARCHAR (Optional)	Institutional identifier used for eligibility verification in private elections such as college elections.
employee_id	VARCHAR (Optional)	Employee identifier used when elections are conducted within corporate or organizational environments.
address_line	TEXT	The user's residential address used for geographic eligibility verification in public elections.
city	VARCHAR	City component of the user's address.
state	VARCHAR	State component of the user's address.
pincode	VARCHAR	Postal code used to determine constituency or geographic eligibility for public elections.
latitude	FLOAT	Geographic coordinate derived from geocoding the user's address. Used for spatial eligibility checks.
longitude	FLOAT	Geographic coordinate derived from geocoding the user's address.
status	ENUM ('active','suspended')	Indicates whether the user account is active or restricted.
created_at	TIMESTAMP	Timestamp recording when the user account was created.

Elections Table

Table Name: `elections`

The **Elections table** stores the core metadata and configuration details for each election created in the system. Every election is associated with a specific organization and contains the parameters required to deploy and manage the election lifecycle.

This table does not store votes themselves; instead, it maintains the configuration used to deploy the election smart contract and track the election within the backend system. The actual votes are recorded on the blockchain ledger through the deployed smart contract.

The table also stores cryptographic references such as the **Merkle root representing the eligible voter list** and the **candidate list hash**, ensuring that election configuration cannot be altered after it has been finalized.

Field Name	Data Type	Description
election_id	UUID (Primary Key)	Unique identifier assigned to each election.
title	VARCHAR	Name of the election (e.g., “Student Council Election 2026”).
description	TEXT	Detailed description explaining the election purpose.
election_type	ENUM ('single_seat','multi_seat')	Specifies whether the election selects one winner or multiple winners.
visibility_type	ENUM ('private','public')	Determines whether the election is restricted to an institution (private) or open to geographic voters (public).

start_time	TIMESTAMP	Timestamp indicating when voting begins.
end_time	TIMESTAMP	Timestamp indicating when voting ends.
eligibility_merkle_root	VARCHAR	Cryptographic Merkle root representing the full eligible voter list for private elections.
location_rule_hash	VARCHAR	Hash representing geographic eligibility rules for public elections.
eligibility_locked	BOOLEAN	Indicates whether the voter eligibility configuration has been finalized and locked.
candidates_locked	BOOLEAN	Indicates whether the candidate list has been finalized and cannot be modified.
candidate_list_hash	VARCHAR	Cryptographic hash representing the finalized candidate list to prevent tampering.
contract_address	VARCHAR	Blockchain address of the deployed election smart contract.
contract_deployed_at	TIMESTAMP	Timestamp recording when the election smart contract was deployed.
created_by_admin	UUID (Foreign Key)	Identifier of the administrator who created the election.
status	ENUM ('draft','scheduled','active','completed')	Current lifecycle stage of the election.

results_published	BOOLEAN	Indicates whether the final election results have been published.
created_at	TIMESTAMP	Timestamp when the election record was created.

Candidates Table

Table Name: `candidates`

The **Candidates table** stores information about individuals contesting in a specific election. Each candidate is associated with an election through the `election_id` field, ensuring that candidates are grouped within the correct election context.

This table contains identity details, ballot display information, and administrative status used to manage candidates during the election configuration process. The table is used by the backend to generate the ballot interface presented to voters.

The table **does not store vote counts**. Votes are recorded on the blockchain through the election smart contract. The database only stores candidate metadata required for election configuration and ballot generation.

The table also includes a **constituency field**, which defines the specific constituency, department, ward, or electoral region the candidate represents within the election. This field becomes particularly important in elections where multiple constituencies exist within the same election.

Field Name	Data Type	Description
candidate_id	UUID (Primary Key)	Unique identifier assigned to each candidate in the system.
constituency_id	UUID (Foreign Key)	References the election to which the candidate belongs.
candidate_name	VARCHAR	Full name of the candidate contesting in the election.

candidate_identifier	VARCHAR	Unique identifier such as Student ID, Employee ID, or other institutional identifier used for validation.
party_name	VARCHAR	Name of the political party, organization, or group the candidate represents.
symbol_url	VARCHAR	URL or file path to the candidate's symbol or logo displayed on the ballot.
profile_photo	VARCHAR	URL or file path of the candidate's profile photograph used in the interface.
manifesto	TEXT	Candidate’s campaign statement or description of goals and policies.
candidate_position	VARCHAR	Position the candidate is contesting for (e.g., President, Secretary).
status	ENUM ('active','withdrawn','disqualified')	Indicates whether the candidate is active, withdrawn, or disqualified.
created_at	TIMESTAMP	Timestamp recording when the candidate entry was created.

Constituencies Table

Table Name: `constituencies`

This table stores the constituencies associated with an election. Each constituency represents a geographic area, department, ward, or electoral region within the election.

Field Name	Data Type	Description
constituency_id	UUID (Primary Key)	Unique identifier assigned to each constituency.
election_id	UUID (Foreign Key)	References the election to which the constituency belongs.
constituency_name	VARCHAR	Name of the constituency (e.g., CSE Department, Ward 24).
description	TEXT	Additional information about the constituency if required.
created_at	TIMESTAMP	Timestamp when the constituency record was created.

VOTE TRANSACTIONS TABLE

Stores **metadata of blockchain vote transactions**.

Table: `vote_transactions`

Field	Type
transaction_id (PK)	UUID
election_id (FK)	UUID
voter_id (FK)	UUID
wallet_address	VARCHAR
candidate_id	UUID
blockchain_tx_hash	VARCHAR
timestamp	TIMESTAMP

Election Voters Table

Table Name: `election_voters`
Purpose: Stores the **eligible voter list for a specific election** (used to generate the Merkle root).

Field Name	Provided By	Explanation
id	System Generated	Unique identifier generated by the backend when the voter entry is created.
constituency_id	(FK)	Taken from the constituency for which the voter list is being uploaded.
voter_identifier	Admin Uploaded	Comes from the CSV file uploaded by the admin (Student ID / Employee ID).

hashed_identifier	System Generated	The system hashes the voter identifier to prepare it for Merkle tree construction.
authorization_status	System Assigned	Initially set automatically to `authorized` when the ID is accepted into the eligibility list.
created_at	System Generated	Timestamp automatically recorded when the record is inserted.